

Dendê Eventos Api

Autores:

Andrew Oliveira Neves

Filipe Oliveira Santos

Guilherme Rodrigues França da Rocha

Gustavo Santos França

José Raimundo de Melo Filho

Vitória da Conquista, 2026

Contextualização



Olá somos trainees da Dendê Softhouse, integrantes do time de Desenvolvimento Web!

1.Desafio(passado) – Refatoração e evolução do código

- Builder para Evento: reduzir complexidade na criação de objetos e evitar construtores extensos/nulos.
- DTOs: proteger o modelo e controlar o que é enviado/recebido nos endpoints.
- Controllers por classe/modelo: organizar rotas conforme o domínio.
- Mappers: converter entre Model ↔ DTO de forma padronizada.

2.Desafio (atual) – Generics e Persistência de Dados

- Incluir das novas dependências do projeto;
- Configuração necessária para persistência de dados com JDBC
- Montar um diagrama de sequência (endpoint: cadastro de usuário comum);
- Refatorar o código anterior, deixando-o mais modularizado e aberto à extensões;
- Adicionar tratamento de excessões quando necessário.

Agenda

Essa apresentação tem como objetivo demonstrar a estrutura, organização e principais implementações da Dendê Eventos API ao refatorarmos o código, adicionando persistência de dados.

1. Dependências:

Bibliotecas e frameworks utilizados no projeto para desenvolvimento da API.

2. Configurações externas:

Arquivos de configuração responsáveis pela conexão e funcionamento do sistema.

3. Persistência com JDBC:

Implementação do acesso e salvamento de dados no banco.

4. Padrões de repositório:

Uso do Repository Pattern para organizar o acesso aos dados.

5. Excessões customizadas:

Tratamento personalizado de erros para melhorar a segurança e organização da API.

6. Refatoração com Lombok:

Redução de código repetitivo utilizando anotações do Lombok.

7. Diagrama de sequência:

Representação do fluxo do cadastro de usuário comum dentro do sistema.

8. Considerações finais:

Resumo dos resultados e benefícios obtidos com o projeto.

9. Referências:

Documentações e materiais utilizados no desenvolvimento do projeto.

Dependências



1. Adição e atualização das Dependências

```
dependencies {  
    testImplementation(platform(notation = "org.junit:junit-bom:5.10.0"))  
    testImplementation("org.junit.jupiter:junit-jupiter")  
    implementation("io.github.lasilva:dendeframework:1.0.2")  
    implementation("com.mysql:mysql-connector-j:8.4.0")  
    implementation("com.zaxxer:HikariCP:5.1.0")  
    compileOnly("org.projectlombok:lombok:1.18.34")  
    annotationProcessor("org.projectlombok:lombok:1.18.34")  
    testCompileOnly("org.projectlombok:lombok:1.18.34")  
    testAnnotationProcessor("org.projectlombok:lombok:1.18.34")  
}
```

Configurações Externas



1.application.properties

Arquivo de configuração com todas as propriedades HikariCP/DataSource

```
datasource.url=jdbc:mysql://localhost:3306/dende_eventos?u  
datasource.username=user  
datasource.password=password  
datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
datasource.hikari.maximum-pool-size=10  
datasource.hikari.minimum-idle=2  
datasource.hikari.connection-timeout=30000
```


Configurações Externas



2.ConfigProperties & @Value

Mapeia properties para Java; mostra anotação @Value do dndeframework e método fromClasspath()

```
@Setter 6 usages 2 Gussfy *
@Getter
public class ConfigProperties {
    // Propriedades do DataSource
    @Value(key = "datasource.url")
    private String datasourceUrl;

    @Value(key = "datasource.username")
    private String datasourceUsername;
```

```
public static ConfigProperties fromClasspath() {
    Properties properties = new Properties();

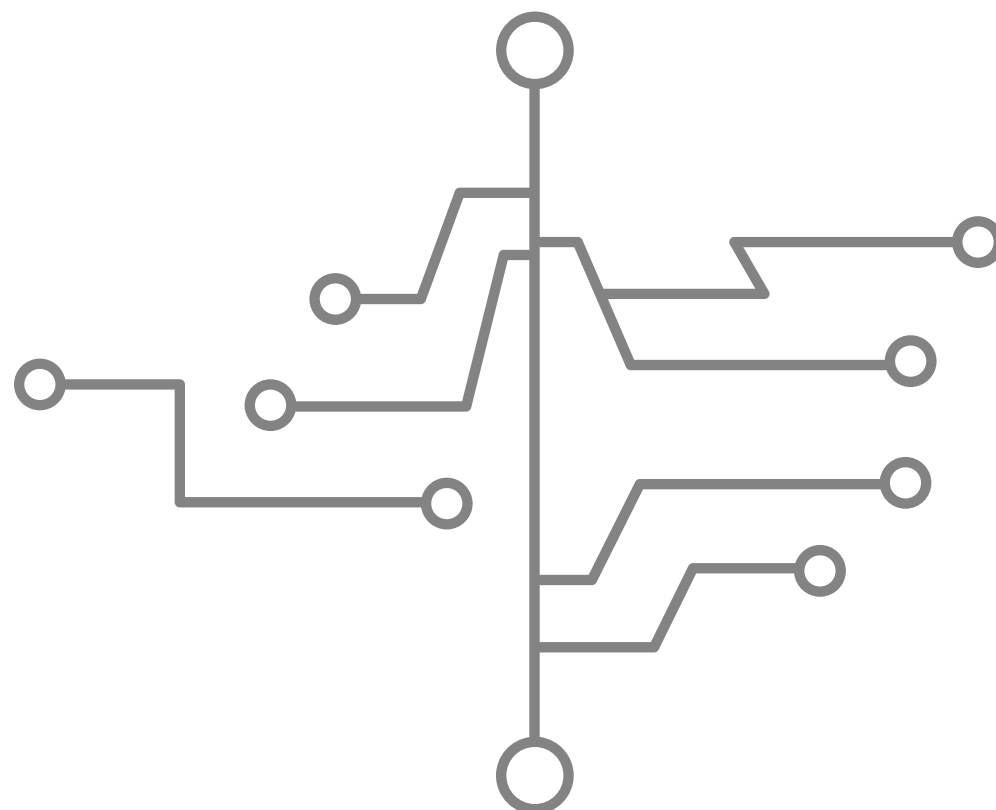
    try (InputStream inputStream = ConfigProperti
        if (inputStream == null) {
            throw new ConfigurationException("Arg
        }
        properties.load(inputStream);
    } catch (IOException e) {
        throw new ConfigurationException("Erro ao
    }
```

Configurações Externas



3.ConnectionPool com HikariCP

Singleton, inicialização, gerência de conexões com HikariConfig



```
@Getter 18 usages  ⚡ Gussffy
public class ConnectionPool {

    // Instância única do pool de conexões (padrão Singleton)
    private static ConnectionPool instance; 3 usages

    // DataSource do HikariCP que gerencia o pool de conexões
    private final HikariDataSource dataSource;

    // Configurações de propriedades da aplicação
    private final ConfigProperties configProperties;

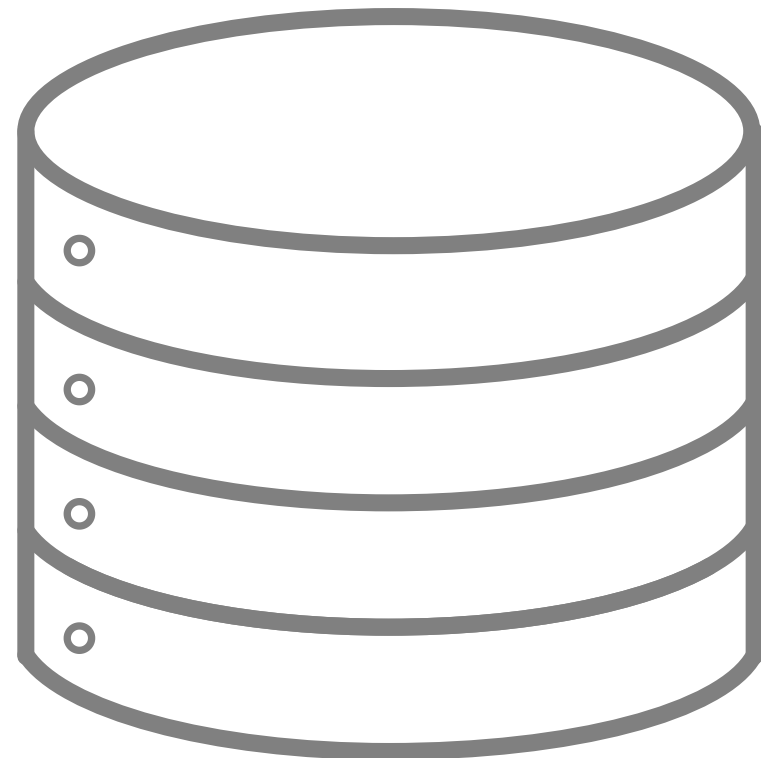
    private ConnectionPool() { 1 usage  ⚡ Gussffy
        this.configProperties = ConfigProperties.fromClasspath();
        this.dataSource = inicializarDataSource();
    }
```

Dendê Eventos API

Persistência com JDBC

1. JDBC sem JPA/Hibernate

Apenas JDBC com interfaces customizadas



CrudRepository Interface

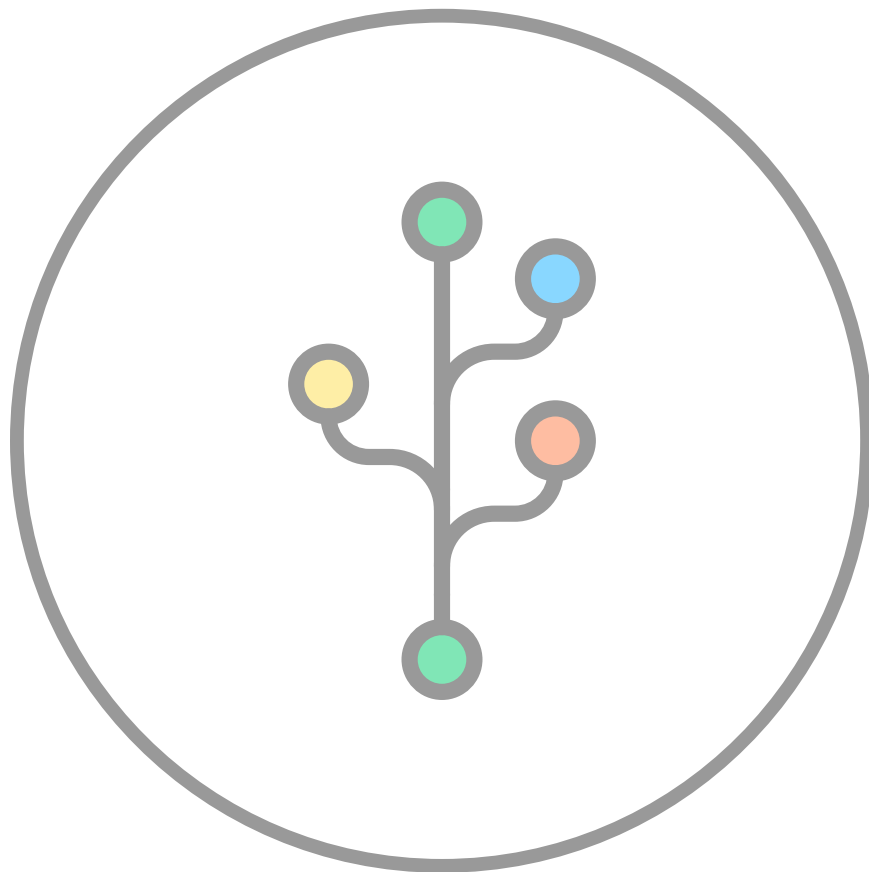
RowMapper Interface

Dendê Eventos API

Padrões de repositório

1. CrudRepository Interface

serve para padronizar e garantir que todos os repositories implementem os mesmos métodos fundamentais



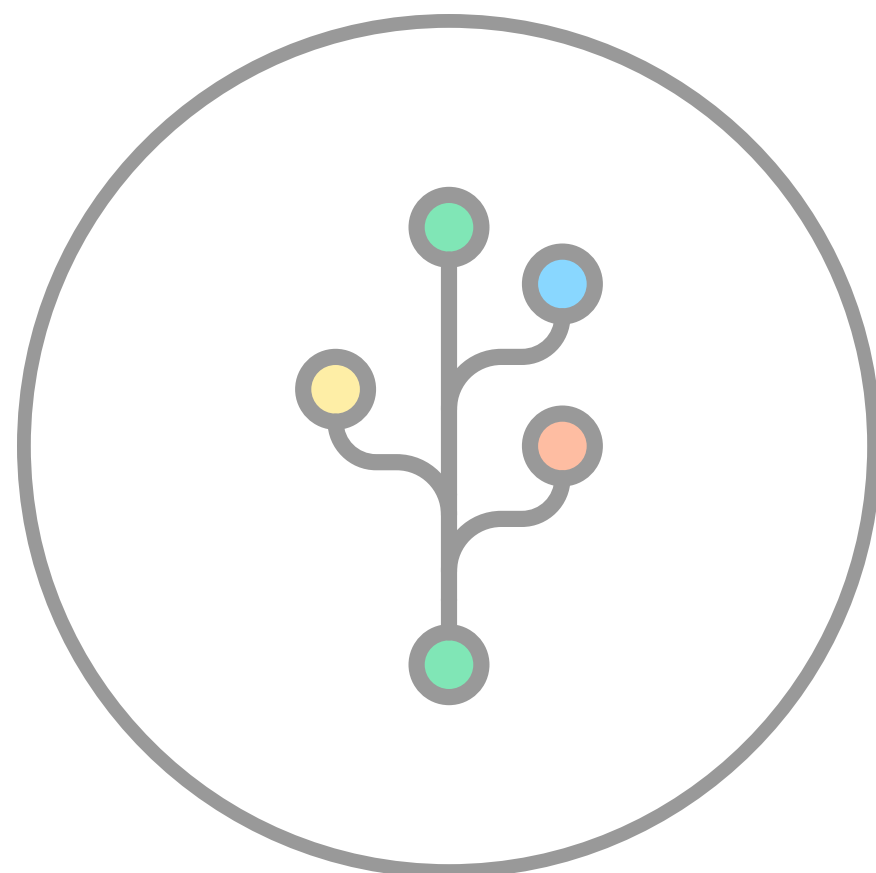
Utilizando Generics

```
public interface CrudRepository<T, ID> { 10 usages 5 implementations Gussffy  
  
    T salvar(T entidade); 5 implementations Gussffy  
  
    void atualizar(T entidade); 5 implementations Gussffy  
  
    T buscarPorId(ID id); 5 implementations Gussffy  
  
    void deletar(ID id); 1 usage 5 implementations Gussffy  
  
    List<T> listarTodos(); no usages 5 implementations Gussffy  
}
```

Padrões de repositório

2.RowMapper Interface

centraliza a conversão de dados brutos do banco (ResultSet/String[]) para objetos Java tipados, garantindo consistência, reutilização e fácil manutenção.



Utilizando Generics

```
package br.com.softhouse.dende.repositories.util;  
  
public interface RowMapper<T> { 10 usages 5 implemen  
  
    T mapRow(String[] row); 5 implementations ⓘ Gussf  
}
```

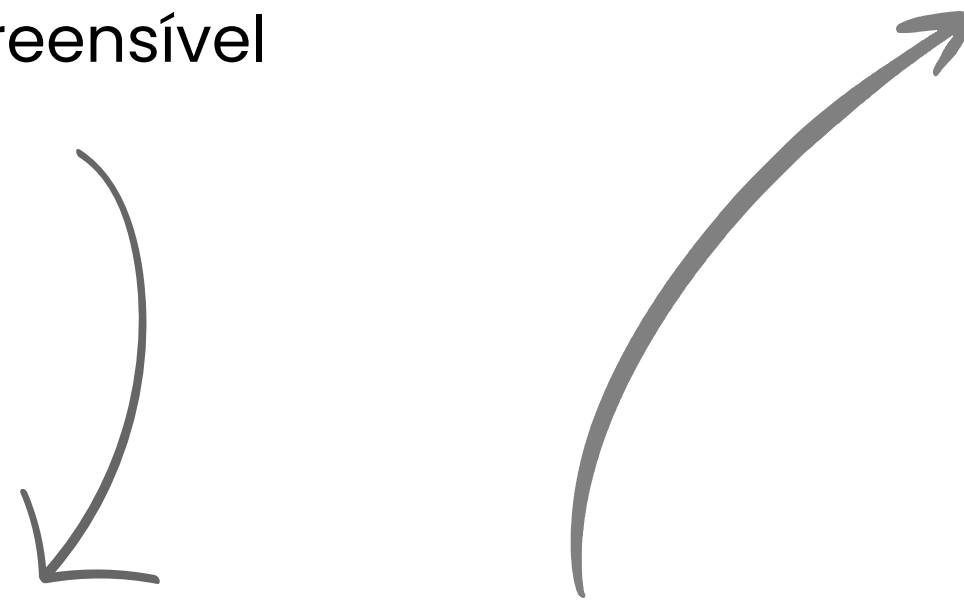


Excessões customizadas

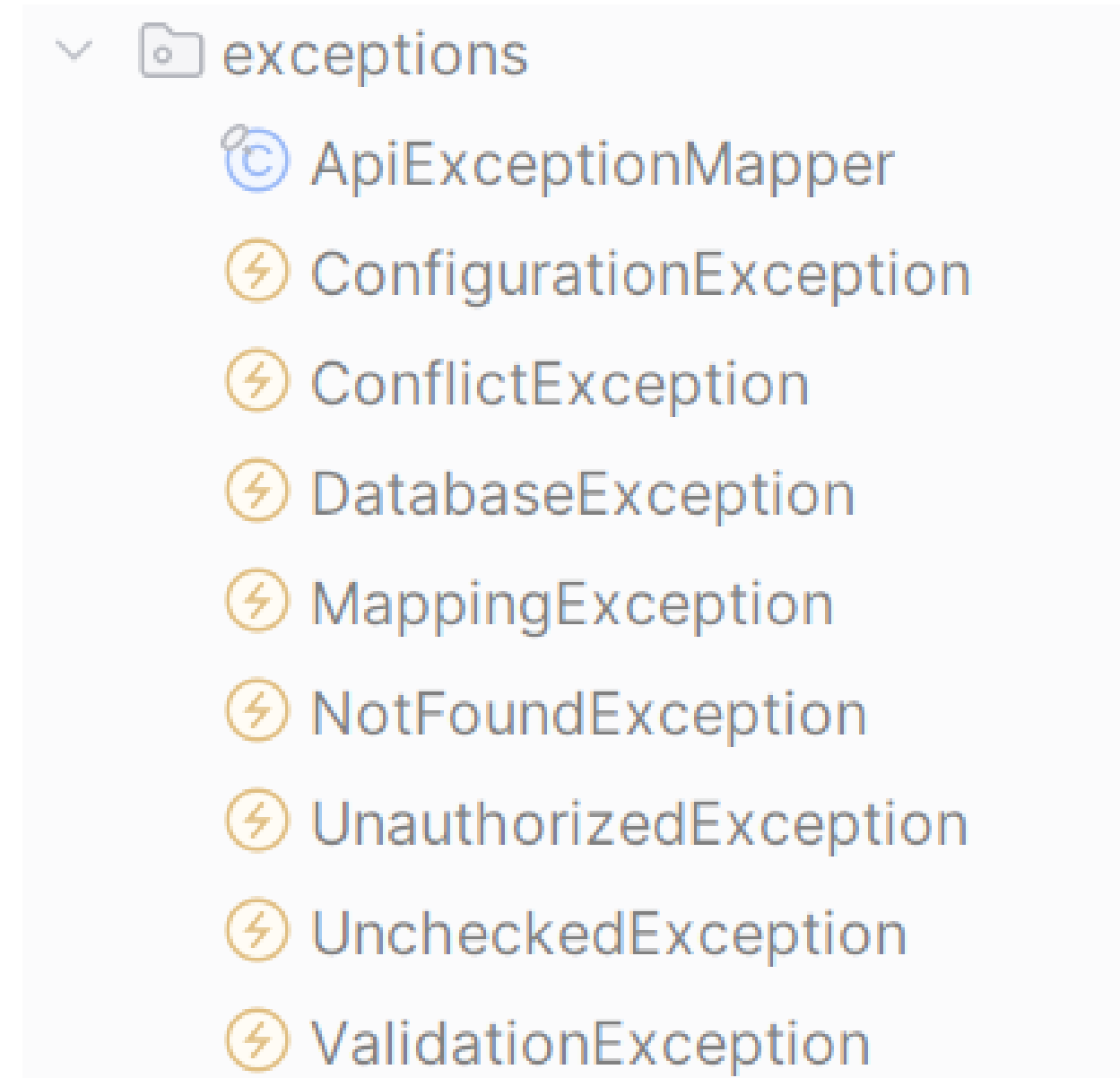


Tratamento personalizado de erros da aplicação

Implementação de exceções personalizadas para tornar o tratamento de erros mais organizado, seguro e compreensível para o usuário e para a API.



- **Exceção raiz: `UncheckedException` extends `RuntimeException`**
- **8 exceções específicas para casos conhecidos**



Refatoração com Lombok



Redução de código boilerplate

Utilização do Lombok para reduzir código repetitivo nas entidades, gerando automaticamente getters, setters, construtores e builders através de anotações.



Antes

```
public Usuario() { 2 usages new *
    this.ativo = true; // Por padrão, o usuário é
}

public Usuario(String nome, LocalDate dataNascimen
    this.nome = nome;
    this.dataNascimento = dataNascimento;
    this.sexo = sexo;
    this.email = email;
    this.senha = senha;
    this.ativo = true;
}

// Getters e Setters
public Long getId() { return id; } new *
public void setId(Long id) { this.id = id; } new *

public String getNome() { return nome; } new *
public void setNome(String nome) { this.nome = nom
```

Depois

```
@Getter 39 usages @ GustavoSantos +2
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Usuario {

    // Atributos do usuário
    private Long id;
    private String nome;
    private LocalDate dataNascimento;
    private Sexo sexo;
    private String email;
    private String senha;
    private Boolean ativo = true; // Por padr

    //Metodo para calcular a idade do usuário
```

Diagrama de sequência

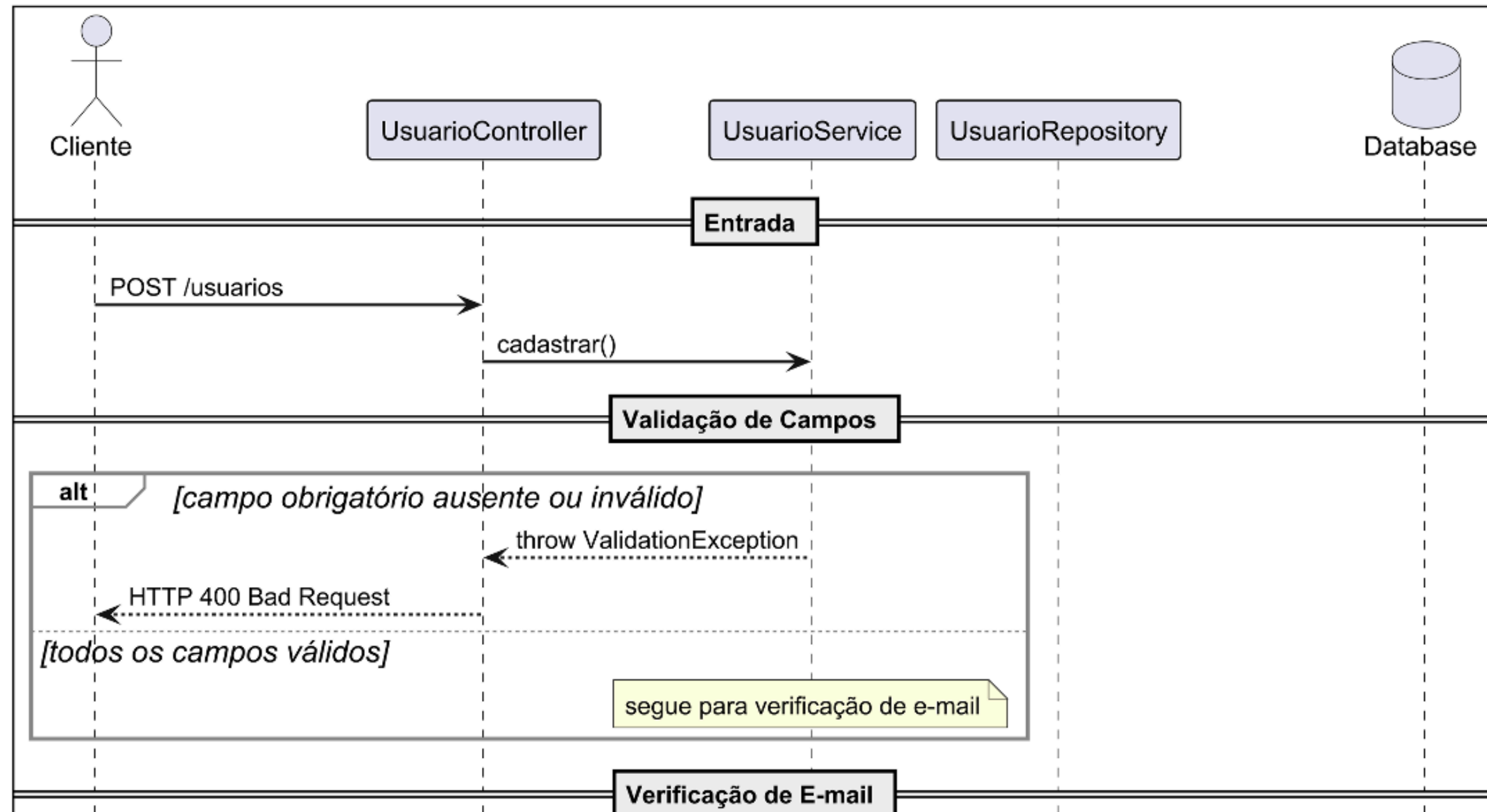
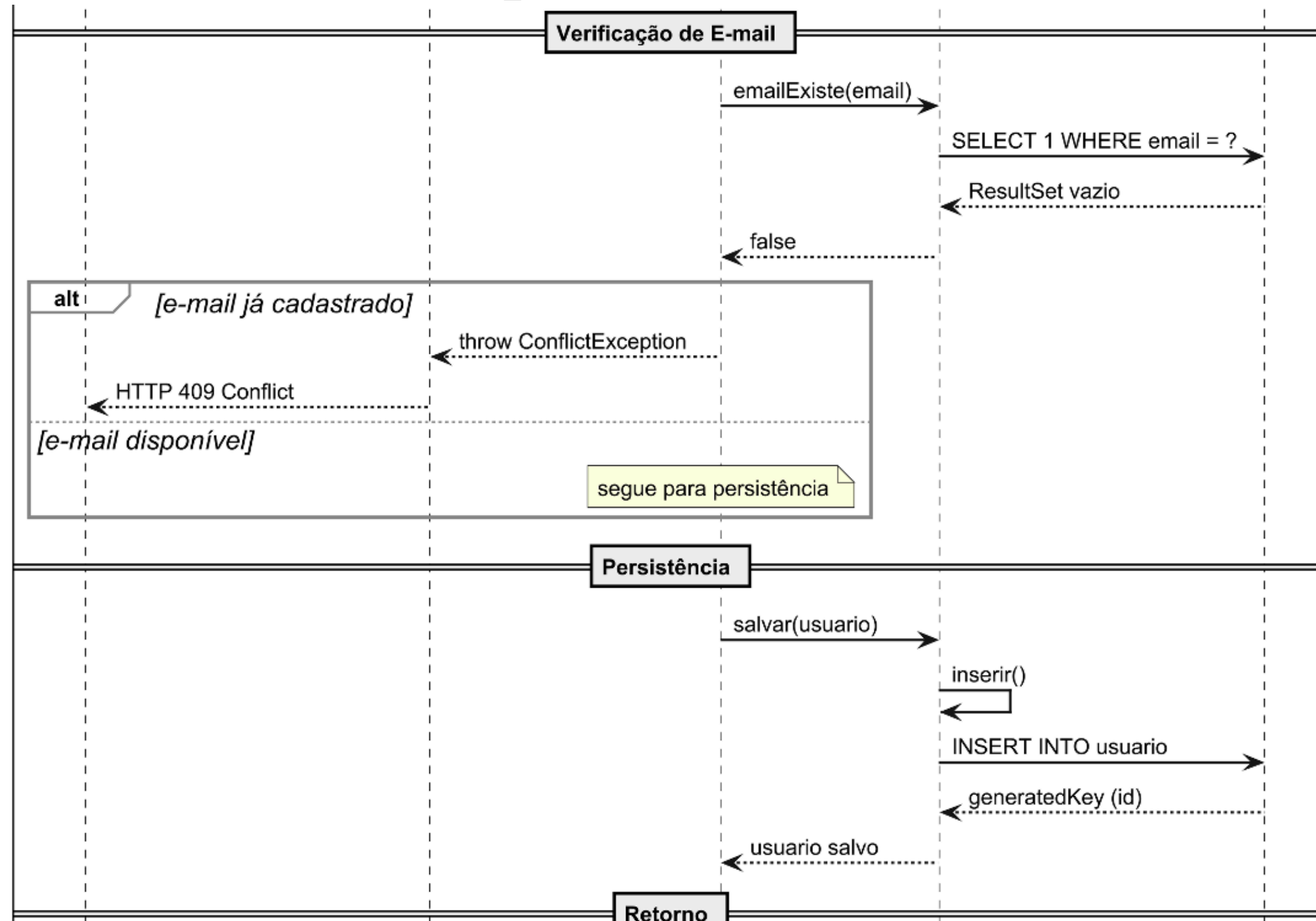
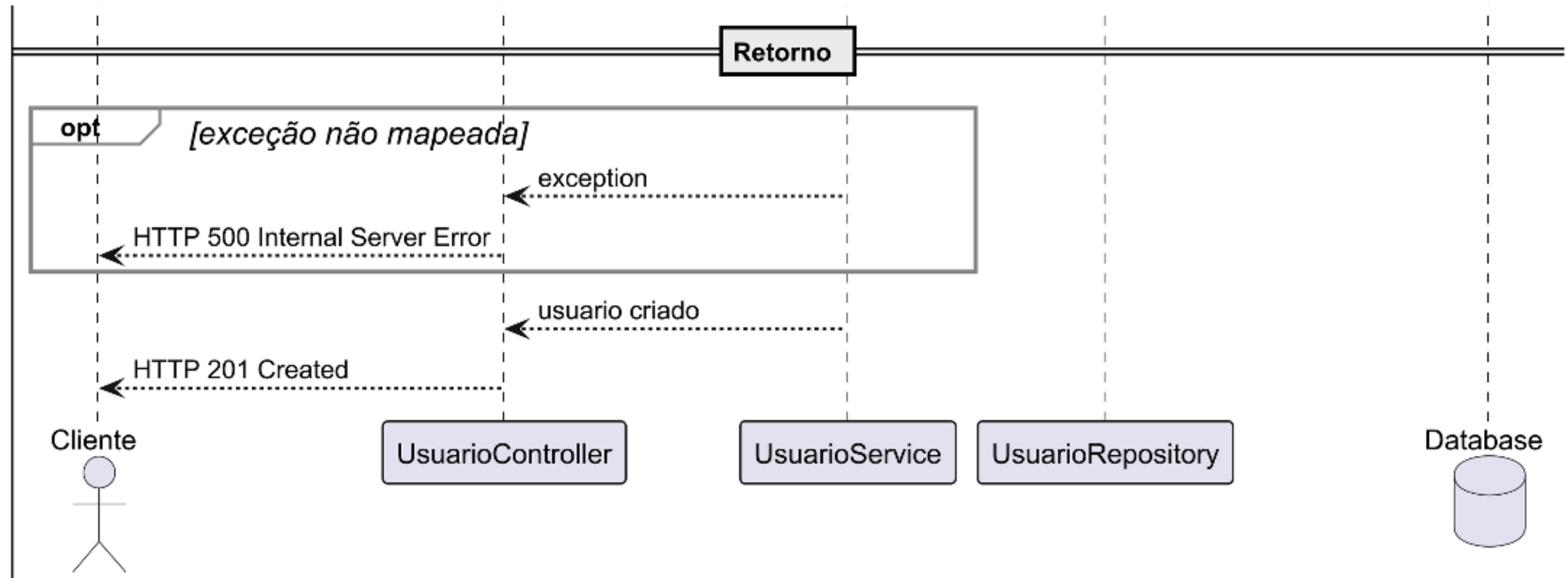


Diagrama de sequência



Dendê Eventos API

Diagrama de sequência



Considerações finais



Dendê Eventos API

Referências



- <https://docs.oracle.com/en/java>
- <https://projectlombok.org/features>
- <https://dev.mysql.com/doc>
- <https://learning.postman.com>